

claude necesito tu ayuda seras mi JARVIS, ahora tengo que resolver este challenge aca te lo adjunto en un pdf y aca esta lo que yo planifique dime que opinas y si hay mejoras de acuerdo al challenge que es en lo que me base para hacerlo, acepto sugerencias, aca lo tienes, LO QUE NECESITO AHORA Quiero empezar a desarrollar el challenge. Arrancamos por "el planning formal" y tambien te adjunto el CSV

```
Claude, Voy a desarrollar un challenge técnico para una posición de Semi-Senior/Senior Python Developer. Necesito que me acompañes durante todo el desarrollo. Acá está todo el contexto:

El CHALLENGE
Desarrollar una API con los siguientes servicios y endpoints:
Data Service
Conectarse a un archivo CSV (se descarga del enunciado)
Endpoint GraphQL con schema completa y documentado
Endpoint NLP consulta en lenguaje natural sobre los datos del CSV, responde en lenguaje natural
Docs Service
Swagger/OpenAPI 3.0 documentando todos los endpoints, con ejemplos reales del dataset
Auth Service (opcional pero lo voy a implementar)
OAuth2 client credentials flow
Retorna JWT
El JWT es requerido como Bearer token en Authorization header para los endpoints del Data Service
Requisitos generales
Todo en Docker
Repositorio público en GitHub con commits progresivos reales (no un commit final)
README explicando cómo compilar y levantar
Se acepta uso de IA pero hay que entregar los chats en PDF

DECISIONES DE ARQUITECTURA YA TOMADAS
Nivel 1 - Distribución del sistema: Microservicios - 3 contenedores Docker separados (auth_service, data_service, docs_service)
Nivel 2 - Organización interna: N-Layer simplificada - router - service - data, Sin Clean Architecture para el Hexagonal (over-engineering para este scope)
Nivel 3 - Patrones: SOLID aplicado naturalmente, Repository pattern para csv_service, Dependency Injection con FastAPI Depends(), DTOs con Pydantic v2

STACK TECNOLÓGICO ELEGIDO
FastAPI para los 3 servicios
Strawberry para GraphQL (tipado con Python type hints, integración nativa con FastAPI)
Claude API (Anthropic) para el endpoint NLP
python-jose para JWT y OAuth2
Pydantic v2 para validación y DTOs
pandas para lectura del CSV
Docker + docker-compose
python-dotenv + .env para configuración

ESTRUCTURA DE CARPETAS DEFINIDA
challenge/
├── auth_service/
│   ├── main.py                (-40 líneas)
│   ├── jwt_handler.py         (-40 líneas)
│   ├── schema.py              (-20 líneas)
│   └── config.py               (-20 líneas)
├── data_service/
│   ├── main.py                (-30 líneas)
│   ├── routers/
│   │   ├── graphql.py         (-50 líneas)
│   │   └── nlp.py              (-40 líneas)
│   ├── services/
│   │   ├── csv_service.py      (-80 líneas - solo carga y expone datos crudos)
│   │   └── nlp_service.py      (-60 líneas - Claude API client)
│   ├── schema/
│   │   ├── types.py           (-60 líneas - Strawberry types)
│   │   └── schemas.py          (-40 líneas - Pydantic DTOs)
│   ├── middleware/
│   │   ├── __init__.py         (-30 líneas - valida Bearer token)
│   │   └── config.py           (-20 líneas)
├── docs_service/
│   ├── main.py                (-40 líneas)
│   └── openapi_spec.py         (-80 líneas - spec + ejemplos reales)
├── data/
├── dataset.csv
├── docker-compose.yml
├── .env.example
└── README.md

REGLAS DE MODULARIZACIÓN
Máximo -150 líneas por archivo
Máximo -30 líneas por función
Si un archivo tiene imports de más de 2 módulos distintos - señal de mala de responsabilidades
csv_service.py solo expone datos crudos - transformaciones específicas van en cada consumer
__init__.py con exports explícitos en cada carpeta para imports limpios

QUÉ NO IMPLEMENTAR
Arquitectura Hexagonal (over-engineering)
Clean Architecture para con use cases (burocracia sin valor)
Base de datos (piden CSV)
Redis / Celery / message queues (no hay justificación)
OOP forzada con clases en FastAPI es funcional, respetar el paradigma)
Tests exhaustivos (no los piden)

QUÉ SI IMPLEMENTAR PARA DESTACAR
NLP con Claude API real (no filtro por keywords) - es lo que diferencia del resto
Swagger con ejemplos reales del dataset (el enunciado lo pide explícitamente)
/health endpoint en cada servicio
.env.example con todas las variables documentadas
Commits atómicos con prefijos feat/fix/chore por feature
README nivel producción: setup, variables, ejemplos curl, decisiones de arquitectura

CONTEXTO PERSONAL
Soy desarrollador Python backend con experiencia en FastAPI, Django, PostgreSQL, Docker, AWS,
Redis/Celery y Claude API. Apunto a rol Semi-Senior / Senior. El challenge hay que entregarlo el lunes.
Voy a desarrollarlo el fin de semana.
return go!r, seed, []
}
```

Perfecto, ya leí el challenge completo y el CSV. Acá va mi análisis detallado — el plan es sólido pero hay puntos críticos que te pueden costar puntos o traerte problemas técnicos reales.

Veredicto general: 85/100 antes de los ajustes

La arquitectura, el stack y el criterio de "qué no implementar" son correctos para el nivel que apuntás. Pero hay **3 problemas reales** que te van a morder durante la implementación si no los resolvés antes.

Problemas críticos

1. docs_service — el problema más serio

Planificaste un contenedor separado para los docs. Bien en concepto, pero **¿cómo va a documentar los otros servicios?**

Tienes dos opciones:

Opción A (recomendada): docs_service hace GET

http://data_service:8001/openapi.json y GET

http://auth_service:8002/openapi.json en startup, los agrega y sirve un Swagger UI unificado. Esto es real microservices y queda muy profesional.

Opción B (más simple): docs_service tiene una spec OpenAPI escrita a mano en `openapi_spec.py` con ejemplos reales del CSV. El enunciado dice "must include examples" — esto cumple igual.

Tu plan actual dice `openapi_spec.py` (~80 líneas – spec + ejemplos reales) lo que sugiere la Opción B, pero entonces no necesitás Docker networking entre servicios para docs. Definí esto ahora o te vas a quedar a mitad durante el finde.

Mi recomendación: Opción A, pero si el tiempo apremia el domingo, Opción B con ejemplos reales queda muy bien igual.

2. NLP + CSV grande — el problema más técnico

El CSV tiene 23 columnas de datos de e-commerce (transacciones con SKUs, dispositivos, fuentes de tráfico, categorías, marcas, métricas de carrito). Es grande. **No podés mandárselo completo a Claude.**

Necesitás una estrategia de contexto. Te propongo esto para `nlp_service.py`:

```
# En vez de raw CSV, construís un contexto inteligente con pandas
def build_csv_context(df: pd.DataFrame) -> str:
    return f"""
Dataset: e-commerce product interactions ({len(df)} rows,
{len(df.columns)} columns)

Columns: {'', '.join(df.columns.tolist())}

Sample data (5 rows):
{df.head(5).to_markdown()}

Key stats:
- Date range: {df['id_tie_fecha_valor'].min()} to
{df['id_tie_fecha_valor'].max()}
- Unique clients: {df['id_cli_cliente'].nunique()}
- Device types: {df['id_ga_tipo_dispositivo'].unique().tolist()}
- Top categories:
{df['glb_d_categoria'].value_counts().head(5).to_dict()}
- Top brands: {df['glb_d_marca'].value_counts().head(5).to_dict()}
"""
```

Esto va como system prompt a Claude, y la pregunta del usuario va como user message. Sin esto, o falla por tokens o las respuestas son hallucinations.

3. Strawberry + JWT middleware — problema silencioso

Strawberry maneja su propio request lifecycle. Si ponés el JWT como middleware HTTP estándar de FastAPI/Starlette, **puede que no intercepte bien el endpoint /graphql**.

La forma correcta es pasarlo via context_getter:

```
# En graphql.py – NO uses middleware HTTP para GraphQL
async def get_context(request: Request, token: str =
Depends(verify_token)):
    return {"request": request, "user": token}

graphql_app = GraphQL(schema, context_getter=get_context)
router.add_route("/graphql", graphql_app)
```

Si usás middleware HTTP genérico, el playground de GraphQL tampoco va a funcionar sin token, lo cual es molesto para la demo.

Ajustes menores pero que suman puntos

GraphQL schema — los nombres de columnas son crípticos

Las columnas del CSV tienen nombres internos (`id_tie_fecha_valor`, `glb_d_categoria`, `id_ga_fuente_medio`). Tu schema de Strawberry debería mapearlos a nombres legibles:

```
@strawberry.type
class ProductInteraction:
    date: str = strawberry.field(name="date") #
id_tie_fecha_valor
    client_id: str = strawberry.field(name="clientId") #
id_cli_cliente
    device_type: str = strawberry.field(name="deviceType") #
id_ga_tipo_dispositivo
    category: str = strawberry.field(name="category") #
glb_d_categoria
    brand: str = strawberry.field(name="brand") #
glb_d_marca
    # etc.
```

También expone queries útiles dado el dataset:

- `productsByCategory(category: String!)`
- `interactionsByDevice(deviceType: String!)`
- `topBrands(limit: Int = 10)`

Dependencias entre contenedores en `docker-compose.yml`

Tu plan no menciona `depends_on` + `healthcheck`. Esto es básico pero visible:

```
data_service:
  depends_on:
    auth_service:
```

```
condition: service_healthy
```

Si no lo tenés, en la demo `data_service` puede arrancar antes que `auth_service` y fallar el primer request.

Una cosa que falta en tu lista de "qué sí implementar"

Error responses consistentes — el enunciado es para Senior. Los evaluadores van a probar requests sin token, con queries GraphQL inválidas, con NLP questions sin sentido. Definí un formato de error estándar:

```
# models/schemas.py
class ErrorResponse(BaseModel):
    error: str
    detail: str
    status_code: int
```

Lo que está perfecto — no toques

- Strawberry sobre Ariadne/graphene-python: decisión correcta para FastAPI nativo
- Claude API real para NLP en vez de keywords: es el diferenciador real
- N-Layer simplificada: perfecto para el scope, evita burocracia
- Pydantic v2: correcto, v1 está deprecated
- python-jose para JWT: correcto
- La regla de 150 líneas por archivo: muy buena disciplina visible en los commits

Orden de implementación sugerido para el finde

Sábado AM: `auth_service` completo + JWT flow testeado con curl

Sábado PM: `csv_service` (carga CSV) + GraphQL schema + queries básicas

Domingo AM: NLP service con contexto inteligente + Claude integration

Domingo PM: `docs_service` + docker-compose completo + README + `.env.example`

Domingo N: commits atómicos limpios, push, chequeo final

Resumen de correcciones

#	Qué	Urgencia
1	Definir estrategia docs_service (Opción A o B)	CRÍTICA
2	Diseñar contexto CSV para Claude (no raw CSV)	CRÍTICA
3	JWT via context_getter en Strawberry, no middleware	CRÍTICA
4	Mapear columnas crípticas a nombres legibles en GraphQL	Media
5	depends_on + healthcheck en docker-compose	Media
6	Formato de error estándar con Pydantic	Baja

El plan base es bueno — con estos ajustes llegás al nivel Senior sin drama. ¿Arrancamos con el auth_service o querés que revisemos el schema GraphQL primero dado el dataset?

crea un archivo reglas.md y pasa el texto planificado con las mejoras que sugieres y elige la opcion A

reglas.md creado. Tiene todo el plan original más:

- **Opción A docs_service** con el código del aggregator listo para copiar
- **Contexto NLP inteligente** con la función build_nlp_context completa
- **JWT via context_getter** para Strawberry con snippet incluido
- **GraphQL schema** con columnas mapeadas y las 5 queries útiles definidas
- **docker-compose** con depends_on + healthcheck snippet
- **ErrorResponse** estándar con Pydantic
- Puertos definidos y .env.example completo

Cuando arranques el sábado, arrancamos con auth_service o el csv_service + schema de Strawberry — lo que prefieras.

bien ahora vamos a crear el planning formal "Voy a desarrollar un challenge técnico para una posición de Semi-Senior/Senior Python Developer en CFOTech. Necesito que me acompañes durante todo el desarrollo este fin de semana. Primero necesito que crees un archivo PLANNING.md en la carpeta del proyecto con el planning formal completo y profesional, listo para entregar al Hiring Manager. DATOS DEL DOCUMENTO

- **Nombre:** Franco
- **Rol aplicado:** Semi-Senior FullStack Python Developer
- **Fecha de inicio:** sábado 22 de marzo de 2026
- **Fecha de entrega:** lunes 23 de marzo de 2026 EL CHALLENGE PIDE
- **Desarrollar una API con 2 servicios y 3 endpoints (opcionalmente 4):**
- **Data Service con endpoint GraphQL y endpoint NLP (consulta en lenguaje natural sobre datos de un CSV)**
- **Docs Service con Swagger/OpenAPI 3.0 con ejemplos**
- **Auth Service opcional con OAuth2 client credentials flow + JWT requerido como Bearer token en todos los endpoints del Data Service Todo en Docker, repositorio público en GitHub con commits progresivos reales, README completo. STACK Y DECISIONES TÉCNICAS**
- **FastAPI para los 3 servicios**
- **Strawberry para GraphQL**
- **Claude API (Anthropic) para el endpoint NLP**
- **python-jose para JWT y OAuth2**
- **Pydantic v2 para validación y DTOs**
- **pandas para lectura del CSV**
- **Docker + docker-compose**
- **3 contenedores separados: auth_service, data_service, docs_service**
- **Arquitectura N-Layer simplificada: router → service → data**
- **Patrones: SOLID, Repository pattern, Dependency Injection con FastAPI Depends() ESTRUCTURA DE CARPETAS challenge/**
 - ├─ auth_service/ |
 - ├─ main.py | └─ jwt_handler.py | └─ schemas.py | └─ config.py └─
 - data_service/ | └─ main.py | └─ routers/ | | └─ graphql.py | | └─
 - nlp.py | └─ services/ | | └─ csv_service.py | | └─ nlp_service.py |
 - └─ schema/ | | └─ types.py | └─ models/ | | └─ schemas.py | └─
 - middleware/ | | └─ auth.py | └─ config.py └─ docs_service/ | └─
 - main.py | └─ openapi_spec.py └─ data/ | └─ dataset.csv └─ docker-
 - compose.yml └─ .env.example └─ README.md
- **ESTIMACIÓN DE TIEMPOS**
- **Setup inicial + Docker: 1h**
- **Auth Service + JWT: 2h**
- **Data Service + CSV loader: 1h**
- **GraphQL endpoint: 3h**
- **NLP endpoint con Claude API: 3h**
- **Swagger + ejemplos reales del CSV: 2h**
- **README + commits + repo: 1h**
- **Total: ~13hs distribuidas en el fin de semana RIESGOS IDENTIFICADOS**

- El endpoint NLP tiene requerimientos ambiguos — decisión tomada: usar Claude API con el CSV como contexto para respuestas en lenguaje natural real
- Los commits deben ser progresivos y reales, no un commit final — requiere disciplina durante el desarrollo
- Tiempo ajustado a un fin de semana — se priorizan endpoints obligatorios sobre optimizaciones opcionales **CRITERIOS DE CALIDAD**
- **SOLID** — cada archivo tiene una sola responsabilidad
- **Máximo ~150 líneas por archivo, ~30 líneas por función**
- Sin secrets hardcoded — todo por variables de entorno
- Error handling con HTTPException custom por servicio
- /health endpoint en cada servicio
- Swagger con ejemplos reales del dataset **FORMATO DEL PLANNING** Redactá el documento en español, tono profesional, con secciones claras. Usá prosa donde corresponda y tablas donde ayuden a la claridad. Que se vea como un documento que un desarrollador Senior entregaría a un Hiring Manager. Incluí al final una sección "Herramientas de IA utilizadas" mencionando Claude (Anthropic) como asistente durante la planificación y el desarrollo. Guardá el archivo como **PLANNING.md** en /ruta/del/proyecto/ — yo te confirmo la ruta exacta antes de que lo crees." si necesito adaptarlo o mejorarlo dimelo, proponme mejoras

Antes de crear el archivo, tengo **3 correcciones importantes** que harían quedar mal el documento ante un Hiring Manager:

1. **Fechas incorrectas** — El 22 de marzo de 2026 es **domingo**, no sábado. El fin de semana real fue sábado 21 + domingo 22, entrega lunes 23. Lo corrijo.
2. **docs_service estructura desactualizada** — El prompt tiene openapi_spec.py (Opción B manual) pero ya decidimos **Opción A** con aggregator.py. Lo actualizo para que sea consistente con reglas.md.
3. **Decisiones técnicas clave ausentes** — Falta mencionar las 3 decisiones críticas que ya documentamos: contexto NLP inteligente con pandas, JWT via context_getter de Strawberry, y el depends_on + healthcheck en Docker. Un Hiring Manager Senior las va a notar.

Creo el archivo con todo ya corregido:

PLANNING.md creado. Las 3 correcciones que apliqué sobre tu borrador original:

Corregido:

- Fecha de inicio: 22 de marzo (domingo) → 21 de marzo (sábado) — el 22 fue domingo

- `docs_service/openapi_spec.py` → `docs_service/aggregator.py` — consistente con Opción A decidida
- Sección de decisiones técnicas expandida con las 3 resoluciones críticas de `reglas.md`

Agregado:

- Tabla de endpoints completa con puertos y queries GraphQL planificadas
- Tabla de riesgos con probabilidad y mitigación concreta (no solo "riesgo identificado")
- Estrategia de commits con ejemplos reales de los mensajes
- Sección "Herramientas de IA" mencionando los usos concretos, no solo que se usó

El documento está listo para entregarle a un Hiring Manager. ¿Arrancamos con el código? Te propongo empezar por `auth_service` — es el más aislado y desbloquearlo primero te permite testear el JWT flow antes de integrarlo en `data_service`.

añade el flujo de trabajo de tareas al planning antes de comenzar en orden que ninguna se interponga a otra

Voy a insertar la sección de flujo de trabajo entre la estimación de tiempos y la estrategia de commits, con el grafo de dependencias incluido.

Listo. Lo que agregué como sección 7 (y renumeré el resto):

- **Grafo de dependencias visual** — ves de un vistazo qué bloquea qué antes de tocar código
- **Tabla de 15 tareas** con archivos exactos, de qué depende cada una y a qué bloquea
- **Tareas paralelizables** identificadas explícitamente (2+3, 6+7, 8+9 no se bloquean entre sí)
- **Regla de avance** — criterio concreto para marcar una tarea como completa antes de avanzar

El orden garantiza que nunca vas a estar esperando que algo esté listo para continuar. ¿Arrancamos con la tarea 1 (setup del repo y estructura)?

TODAS LAS TAREAS COMPLETADAS!

(Respecto al uso de IA: Claude fue utilizado como herramienta de asistencia técnica, de forma similar a como se usaría un pair programmer o un rubber duck avanzado. Cada decisión de arquitectura, stack y diseño fue tomada por mí con criterio propio y validada o ajustada en base al resultado. La conversación adjunta refleja ese proceso: planteo el problema, evalúo las opciones y decido. Claude ejecuta, yo dirijo.

Algunas secciones de la conversación no se incluyen por contener información sensible como API keys y credenciales utilizadas durante las pruebas locales.)